

Depth, Breadth, or Modularity? A Parameter-Matched Benchmark of Hybrid Neural Architectures for Inference Latency and Accuracy

Aryan Kulkarni

Bellarmino College Preparatory

San Jose, CA, USA

aryanamol10@gmail.com

Abstract—Architecture comparisons often use parameter counts or FLOPs as proxies for speed, but deployed systems are judged by latency. We benchmark six small neural architectures, three single-paradigm models (sequential, parallel, modular) and three hybrids combining depth, breadth, and modularity, all held to 385 parameters ($\pm 0.5\%$) and evaluated on three regression tasks with five seeds. A pilot study suggested a $1.62\times$ speedup for modular networks. An examination traced it to a cross-framework confound and uninformative inputs, and we present it as a case study in benchmarking pitfalls. In the corrected study, with standardized targets, every topology reaches the noise floor on simple tasks. On a composition task, a staged modular hybrid lowers test error by 11% relative to a sequential baseline ($p < 0.05$), a gap that persists after 300 epochs, at $3.3\times$ the single-sample latency. Across models, single-sample latency is almost perfectly predicted by tensor operations per forward call ($R^2 = 0.98$) rather than by parameter count. Compilation narrows this spread ($3.8\times$ to $1.6\times$) but adds a fixed per-call cost. At large batch sizes the ranking inverts in favor of narrow-activation designs. Small models should therefore be compared at the deployment batch size, with operator counts reported.

Index Terms—neural network architecture, hybrid architectures, modular networks, inference latency, benchmarking, efficient machine learning, reproducibility

I. INTRODUCTION

Choosing a neural architecture means choosing how to spend a fixed budget of parameters and compute. The same budget can buy depth (layers in sequence), breadth (parallel branches), or modularity (sub-networks that each handle part of a problem). Modern systems increasingly mix all three of these types: multi-branch blocks [1], [2], shared trunks with task heads [3], and mixtures of experts [4]. Theory says depth and width trade off in representational power [5], [6]. People working on this, however, also care about a different concept: *how fast does the model answer?*

Latency is notoriously hard to predict from the numbers that architecture papers usually report. Parameter count and FLOPs can disagree with the measured speed and can even reverse model rankings [7], [8]. The gap is widest for small models, such as those in embedded control, sensor fusion, or TinyML [9], where fixed per-operator overhead, rather than arithmetic, can dominate run time [10]. Benchmarks of such models are also easy to get wrong. Framework choice, seed variance, and evaluation shortcuts can each produce differences larger than the effect being studied [11], [12].

This paper asks: *at a fixed parameter budget, does the arrangement of parameters (sequential, parallel, modular, or a hybrid) change accuracy and inference latency, and what predicts the latency?* We answer with a controlled benchmark of six architectures at 385 parameters ($\pm 0.5\%$) on three synthetic regression tasks of increasing structural complexity. Synthetic tasks let us know the irreducible error exactly and let us design one task whose compositional structure a modular model could benefit from.

Contributions.

- A *parameter-matched benchmark* of three single-paradigm and three hybrid MLP architectures, with five seeds, held-out evaluation, and a null control (two functionally identical models), released with all code and raw results.
- An *audited pilot study* showing how a cross-system comparison produced a false $1.62\times$ “modular speedup”, and three low-cost practices that catch such errors.
- Evidence that *structure helps only when the task has structure*. All topologies tie on simple targets, but a staged modular hybrid reduces error on a composed target by 11%, a gap that persists after 300 epochs of training.
- A latency analysis showing that, for small models, *single-sample latency is predicted by operator count* ($R^2 = 0.98$) rather than parameters or MACs, that `torch.compile` narrows but does not remove these differences, and that the latency ranking *depends on batch size*.

Section II reviews related work, Section III describes the setup, Section IV audits the pilot, Section V presents results, and Section VI discusses implications and limitations.

II. RELATED WORK

Depth versus breadth. A single hidden layer of sufficient width can approximate any continuous function [13], yet deep networks represent some functions far more efficiently than shallow ones [5], and width has its own limits [6]. We ask the practical question that follows: at a fixed, small parameter budget, which arrangement trains to the lowest error, and which runs fastest?

Multi-branch, shared-trunk, and modular designs. Parallel branches are central to Inception [1] and ResNeXt [2], and

wide residual networks trade depth for width [14]. A shared trunk with several heads is the hard-parameter-sharing pattern of multi-task learning [3], and averaging several predictions underlies deep ensembles [15]. Mixtures of experts route inputs to specialized sub-networks [4], [16], neural module networks mirror the compositional structure of a query [17], and whether networks exploit compositional structure at all remains open [18], [19]. H-B tests the simplest version of this idea: one module per stage of a known composition, joined by dense skip connections [20], [21].

Measuring efficiency. Parameter count and FLOPs are poor proxies for latency: ShuffleNet V2 derived design guidelines from measured speed [7], the choice of cost indicator can reverse model rankings [8], and hardware-aware search therefore optimizes measured latency [22]. Benchmarks such as MLPerf Inference [23], MLPerf Tiny [9], and DAWNBench [24] fix the measurement rules, and surveys explain where time goes in DNN inference [10]. Benchmarks are also fragile: framework choice alone changes measured speed [11], seed variance can exceed the effects being claimed [12], and empirical ML has a history of overstated conclusions [25].

Positioning. We study sub-kilobyte MLPs on a single CPU thread, where overhead is visible in isolation. Unlike capacity comparisons, we keep parameters constant and change only structure.

III. EXPERIMENTAL SETUP

A. Synthetic Regression Tasks

We generate $N = 5,000$ evenly spaced inputs $x \in [0, 10]$ and three targets of increasing structural complexity (Fig. 1), each with additive Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$:

$$y_{\text{lin}} = 2x + \varepsilon, \quad \sigma = 1, \quad (1)$$

$$y_{\text{curv}} = 0.5x^2 + \sin x + \varepsilon, \quad \sigma = 2, \quad (2)$$

$$y_{\text{chain}} = s_a + \sin(s_a) + 0.5 \operatorname{sgn}(\sin x) + \varepsilon, \quad \sigma = 0.5, \quad (3)$$

where $s_a = x \cos(0.5x)$. The *chained* target is a composition of three stages: an oscillating ramp, a nonlinearity applied to that ramp, and a discontinuous square wave. This was designed so that a model whose structure mirrors the stages could, in principle, utilize it. Because the noise is known, each task has an irreducible test MSE of σ^2 (1, 4, and 0.25, respectively), which we use as an absolute reference. Inputs are scaled to $[0, 1]$. Targets are standardized with the training set’s mean and standard deviation, the models are trained on the standardized values, and predictions are mapped back to original units, so every reported MSE is directly comparable to the noise floors. A single random 80/20 train/test split (seed 42) is shared by every model and task, giving 4,000 training and 1,000 test points.

B. Architectures

All six models (Fig. 2) are one-input, one-output multilayer models with ReLU activations, each with exactly 385 trainable parameters, so that differences in accuracy or speed cannot be attributed to capacity. The one exception is H-C: no width

TABLE I
ARCHITECTURE COST PROFILE. DEPTH: LINEAR LAYERS ON THE LONGEST PATH. OPS / KERN.: PYTORCH OPERATIONS / ATEN KERNELS PER FORWARD CALL. LATENCY: MEDIAN SINGLE-THREAD CPU TIME OVER 10 SHUFFLED REPEATS; COMP.: WITH `TORCH.COMPILE`.

Model	MACs/			Latency (μ s)				
	Params	sample	Depth	Ops	Kern.	b=1	comp.	4096
Sequential	385	256	2	3	22	11.7	21.3	196
Parallel	385	256	2	6	35	16.1	25.6	314
Modular	385	256	2	3	22	10.0	20.9	193
H-A	385	336	3	10	59	31.7	22.5	213
H-B	385	286	6	13	70	38.2	33.2	322
H-C	387	350	3	8	50	25.0	25.1	156

pair for two-head wiring totals 385, so it uses the nearest configuration, 387 (+0.5%). Table I lists their parameter counts, multiply-accumulate operations (MACs) per sample, critical-path depth (linear layers between input and output), tensor operations dispatched per forward call, and the low-level (ATen) kernels those operations execute.

Single-paradigm baselines. *Sequential* is a 1–128–1 network. *Parallel* feeds the input to two 64-unit branches whose activations are concatenated before a linear read-out, in the spirit of multi-branch designs such as Inception [1]. *Modular* wraps a hidden layer and read-out in a self-contained module. With 128 hidden units it calculates exactly the same function class as *Sequential*, and under the same seed it initializes to the same weights. We keep it as a *control*: any measured difference between *Sequential* and *Modular* is implementation overhead or timing noise, never architecture.

Hybrids. **H-A** (*parallel of sequentials*) runs two 2-layer, 12-unit stacks side by side, combining depth and breadth. **H-B** (*staged modular chain*) has three sub-networks M_A , M_B , M_C (hidden widths 33, 32, and 31, chosen to total exactly 385 parameters) that mirror the three stages of the chained target: M_B receives (x, a) , M_C receives (x, a, b) , and the output is $a + b + c$. This gives densely connected skip links in the spirit of DenseNet [21] and an additive output similar to residual learning [20]. **H-C** (*trunk + parallel heads*) shares a 2-layer trunk (20 and 15 units) between two linear heads whose outputs are averaged, the hard-parameter-sharing pattern from multi-task learning [3]. No set of architectures can match parameters, MACs, and operation count at once. Fixing parameters forces the other two to vary, and that difference is what lets us test which cost measure actually predicts latency.

C. Training Protocol

Every model is implemented and trained in PyTorch [26] with one shared training loop. We use Adam [27] (learning rate 10^{-3} , $\beta = (0.9, 0.999)$, $\epsilon = 10^{-7}$, matching the Keras default used in the pilot), mean-squared-error loss, mini-batches of 32 with per-epoch shuffling, and 100 epochs (12,500 optimizer steps). To test whether the chained-task results reflect under-training, we also train *Sequential*, *Parallel*, *H-A*, and *H-B* on the chained task for 300 epochs under the same protocol. Each

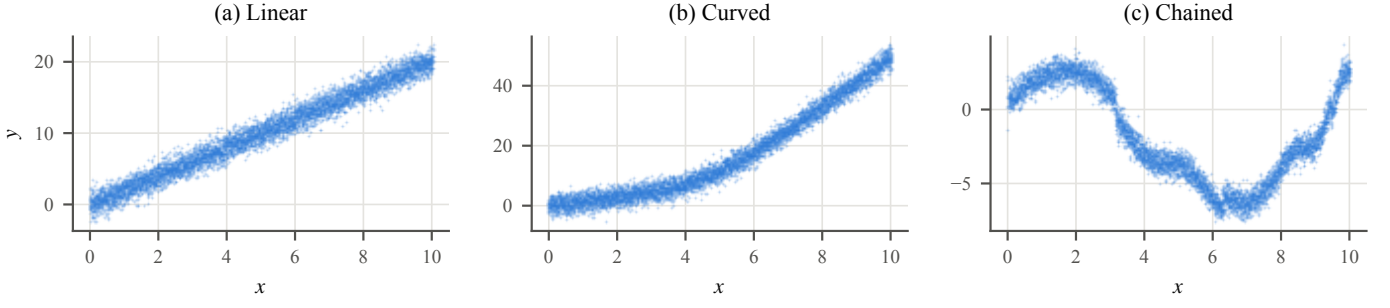


Fig. 1. The three synthetic regression tasks ($N = 5,000$). (a) linear with $\sigma = 1$; (b) quadratic plus sinusoid with $\sigma = 2$; (c) a three-stage composition containing a square-wave discontinuity, $\sigma = 0.5$.

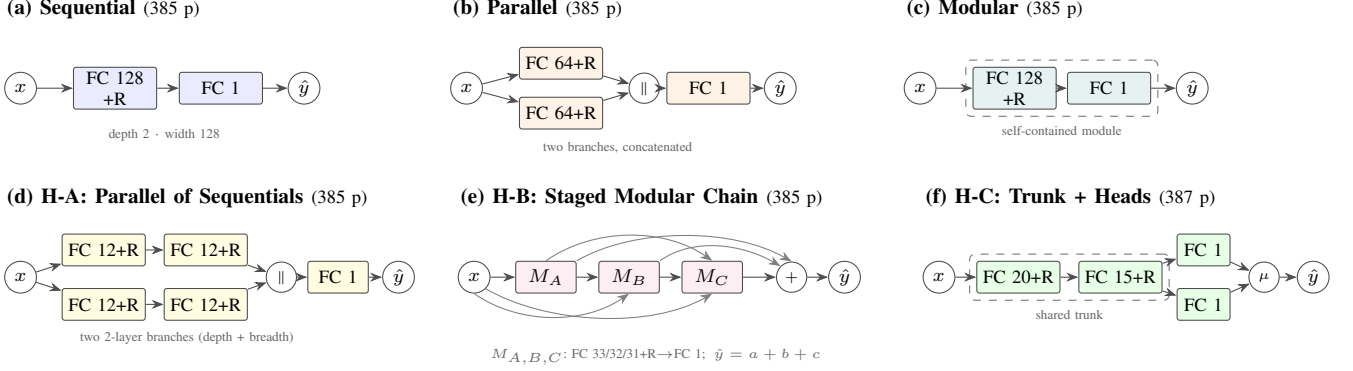


Fig. 2. The six parameter-matched architectures (385 trainable parameters; 387 for H-C). Top row: the three single-paradigm baselines from the pilot study. Bottom row: hybrids combining depth, breadth, and modularity. FC n = fully connected layer with n outputs, R = ReLU, $\parallel$ = concatenation, μ = mean. Note that (a) and (c) compute the same function class; they differ only in how the code is organized, which lets us use them as a control pair for implementation overhead (Section IV).

(model, task) pair is trained with five seeds (0–4) that control initialization and batch order. We report mean $\pm$ standard deviation across seeds, following recommendations to account for training variance in benchmark comparisons [12].

D. Metrics

- **Test MSE** and R^2 on the 1,000 held-out points after the final epoch.
- **Convergence epoch**: the first epoch whose test MSE is within 5% of that run’s best.
- **Training cost**: wall-clock training time divided by optimizer steps (ms/step).
- **Inference latency**: median of 50 forward passes over the full test set (batch 1,000).

Significance against the Sequential baseline uses Welch’s two-sample t -test on test MSE ($n = 5$ per arm, $p < 0.05$). One timing group (H-B, chained task) was re-measured after a single seed’s inference time deviated by 30% from the same network on the other tasks; its accuracy was bit-identical across both passes.

E. Latency Sweep

Real deployments rarely run inference at exactly the batch size used for evaluation, so we also time every model at batch sizes $\{1, 8, 64, 512, 1000, 4096\}$ with 1 and 4 CPU threads.

Each configuration gets 50 warm-up calls followed by 1,000 timed calls, and the whole grid is repeated 10 times in a freshly shuffled order so that run order and cache state cannot result in bias towards any model. We report the median of the per-repeat medians, with a 95% interval across repeats. MACs are counted with forward hooks on every linear layer. Dispatched operations are counted by intercepting PyTorch’s `__torch_function__` protocol during one forward pass, and the kernels those operations execute are counted with `torch.profiler`. We also count `nn.Module` calls with forward hooks. Finally, we time batch-1 inference after compiling each model with `torch.compile` (default Inductor backend), using the same repeat protocol. These sweep timings are separate from the batch-1,000 inference times in Table II, which are measured on the trained models inside the training study; the two agree where they overlap.

F. Environment

All main-study and sweep measurements run on a single machine (Apple M5 CPU, macOS, Python 3.9, PyTorch 2.8.0) with `torch.set_num_threads(1)` unless stated otherwise, so the timings compare architectures rather than thread scheduling. No GPU is used: at this model scale, kernel-launch overhead would dominate any GPU measurement [10]. All code and raw

results are available at <https://github.com/aryanamol10/Research-Paper-Comparative-Analysis-of-Neural-Models>.

IV. PILOT STUDY: HOW A BENCHMARK CAN MISLEAD

Before the main study, we ran a pilot benchmark (Benchmark 1) of the three single-paradigm models in a Google Colab notebook. Its headline result was striking: the Modular network trained about $1.62\times$ faster than Sequential and Parallel on every task (roughly 18 s vs. 30 s per 100-epoch run), and its accuracy was only marginally worse. Taken at face value, this suggests that modularity is a free latency win. Auditing the pilot showed that the result has nothing to do with architecture. We report the audit because each flaw is common in small-scale benchmarking [12], [25].

Confound 1: framework, not architecture. Sequential and Parallel were built in Keras/TensorFlow using `model.fit`, while Modular was a hand-written PyTorch loop. Modular and Sequential compute the same 1–128–1 function, so the ~ 11 s gap is entirely framework and training-loop overhead. Framework-level differences of this size are well documented [11]. They show that any timing comparison across frameworks says little about architecture.

Confound 2: uninformative inputs. The pilot trained on $x \sim \mathcal{U}(0,1)$ drawn independently of the targets, which were computed on the evenly spaced grid. With no information in the inputs, the best any model can do is predict the target mean, which gives an MSE equal to the target variance. The reported losses (37.8, 240, 10.4) sit just above those variances (34.7, 228.8, 10.0). In other words, none of the pilot models learned the task, so their accuracy ranking is noise.

Confound 3: no held-out data, one seed. Losses were final-epoch *training* losses from a single seed with shuffling disabled, so neither generalization nor run-to-run variance could be assessed.

The main study (Section III) fixes all three: one framework and one training loop, true inputs, a held-out split, and five seeds. The identical Sequential/Modular pair stays in as a built-in check. If the protocol is sound, the two must agree on accuracy and differ only by noise.

V. RESULTS

Table II reports accuracy and cost for every (model, task) pair, and Fig. 3 follows the chained task to 300 epochs. The latency sweep is shown in Figs. 4 and 5.

A. The Control Pair: Identical Math, Different Code

Sequential and Modular compute the same function and, under the same seed, produce the same test MSE on every task (Table II). Their training step times differ by only 1%, confirming that the pilot’s $1.62\times$ “Modular advantage” (Section IV) disappears once the framework and training loop are held constant. Their batch-1 inference latency, however, differs by 17% (11.7 vs. 10.0 μ s), with non-overlapping 95% intervals across ten shuffled repeats, so the gap is not run-order noise. Both dispatch three tensor operations, but Sequential’s `nn.Sequential` container makes 5 `nn.Module` calls per

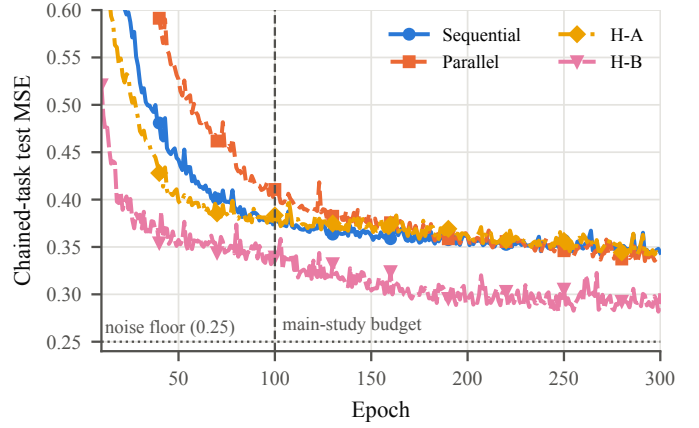


Fig. 3. Chained-task test MSE from epoch 10 to 300 (mean of five seeds). The dashed line marks the 100-epoch budget of the main study. H-B stays lowest throughout; Parallel catches up with Sequential.

forward pass against Modular’s 3. The control pair therefore isolates a second, purely software overhead, showing that code organization costs time even when the math is the same (Section V-C).

B. Accuracy: Structure Pays Off Only When the Task Has Structure

Linear and curved tasks. Every model reaches the noise floor on the linear task (MSE 1.02–1.03 against a floor of 1, $R^2 \approx 0.97$), and on the curved task every model finishes within 2% of the floor of 4. Simple targets leave nothing for topology to improve upon, so at a 385-parameter budget the arrangement of parameters does not change the final accuracy. With standardized targets, every model also converges quickly, within 10 epochs on either task.

Chained task. Here topology matters, but only for the hybrid built for it. H-B, whose modules mirror the three stages of the target, reaches 0.339 test MSE, 11% below Sequential (0.382; $p < 0.05$), and its excess error over the noise floor of 0.25 (0.089) is $1.5\times$ smaller than Sequential’s (0.132). H-A, which has no knowledge of the stages, only matches Sequential (0.383), and plain breadth does not help: Parallel is the worst model (0.411).

Undertraining or a ceiling? At 100 epochs the chained-task models are still improving, so we trained Sequential, Parallel, H-A, and H-B for 300 epochs (Fig. 3). Every model improves: Sequential falls 10% to 0.344, and Parallel (0.338) catches up with it. The H-B advantage persists: it ends at 0.301, 12% below Sequential ($p = 0.005$), with $1.8\times$ smaller excess error. Gains level off late (the last 1% improvement comes after epoch 227 for H-B and 270 for Sequential, on average), so these values are close to each model’s ceiling at this budget. The gap is not an artifact of the 100-epoch training budget.

C. Latency: Operator Count, Not Parameter Count

Table I and Fig. 4 show that, at a nearly constant parameter budget, single-sample inference latency spans almost $4\times$, from

TABLE II

MAIN RESULTS (MEAN $\pm$ STD OVER 5 SEEDS; HELD-OUT TEST SET OF 1,000 POINTS). IRREDUCIBLE TEST MSE IS 1, 4, AND 0.25 FOR THE LINEAR, CURVED, AND CHAINED TASKS. BOLD = BEST AT SHOWN PRECISION. [†]WELCH’S *t*-TEST VS. SEQUENTIAL, $p < 0.05$. INFERENCE = FORWARD PASS OVER THE WHOLE TEST SET (BATCH 1,000), SINGLE CPU THREAD.

Model	Test MSE $\downarrow$	$R^2 \uparrow$	Train (ms/step) $\downarrow$	Inference (ms) $\downarrow$	Converge (epoch) $\downarrow$
<i>Linear task</i>					
Sequential	1.022 $\pm$ 0.026	0.970	0.277 $\pm$ 0.006	0.081 $\pm$ 0.001	2 $\pm$ 0
Parallel	1.023 $\pm$ 0.028	0.970	0.345 $\pm$ 0.012	0.091 $\pm$ 0.000	2 $\pm$ 0
Modular	1.022 $\pm$ 0.026	0.970	0.276 $\pm$ 0.003	0.079 $\pm$ 0.002	2 $\pm$ 0
H-A	1.016 $\pm$ 0.020	0.971	0.425 $\pm$ 0.012	0.093 $\pm$ 0.001	3 $\pm$ 1
H-B	1.027 $\pm$ 0.032	0.970	0.479 $\pm$ 0.009	0.130 $\pm$ 0.001	2 $\pm$ 1
H-C	1.021 $\pm$ 0.027	0.970	0.372 $\pm$ 0.007	0.066 $\pm$ 0.000	8 $\pm$ 9
<i>Curved task</i>					
Sequential	4.045 $\pm$ 0.060	0.982	0.272 $\pm$ 0.005	0.081 $\pm$ 0.000	8 $\pm$ 1
Parallel	4.032 $\pm$ 0.051	0.982	0.326 $\pm$ 0.005	0.091 $\pm$ 0.000	8 $\pm$ 2
Modular	4.045 $\pm$ 0.060	0.982	0.273 $\pm$ 0.007	0.079 $\pm$ 0.001	8 $\pm$ 1
H-A	3.987 $\pm$ 0.019	0.983	0.416 $\pm$ 0.006	0.093 $\pm$ 0.001	8 $\pm$ 3
H-B	4.029 $\pm$ 0.033	0.982	0.480 $\pm$ 0.004	0.129 $\pm$ 0.002	4 $\pm$ 1
H-C	3.994 $\pm$ 0.007	0.983	0.379 $\pm$ 0.009	0.067 $\pm$ 0.000	10 $\pm$ 5
<i>Chained task</i>					
Sequential	0.382 $\pm$ 0.007	0.962	0.273 $\pm$ 0.005	0.082 $\pm$ 0.000	77 $\pm$ 3
Parallel	0.411 $\pm$ 0.071	0.959	0.328 $\pm$ 0.008	0.091 $\pm$ 0.001	81 $\pm$ 8
Modular	0.382 $\pm$ 0.007	0.962	0.265 $\pm$ 0.001	0.078 $\pm$ 0.000	77 $\pm$ 3
H-A	0.383 $\pm$ 0.033	0.961	0.417 $\pm$ 0.004	0.094 $\pm$ 0.000	50 $\pm$ 17
H-B	0.339 $\pm$ 0.013[†]	0.966	0.494 $\pm$ 0.018	0.131 $\pm$ 0.001	54 $\pm$ 19
H-C	0.378 $\pm$ 0.021	0.962	0.371 $\pm$ 0.002	0.067 $\pm$ 0.000	69 $\pm$ 15

10.0 μ s (Modular) to 38.2 μ s (H-B). Parameter count cannot explain this: all models have 385–387 parameters. Neither can MACs: Parallel and Sequential have the same 256 MACs, but Parallel is slower, and H-C has the most MACs (350) but is faster than H-A and H-B. What does explain it is the number of tensor operations dispatched per forward call. A linear fit of batch-1 latency on operation count gives $R^2 = 0.98$ with a slope of about 2.8 μ s per operation. At this scale, each Linear, ReLU, cat, or add pays a fixed framework dispatch cost that dwarfs its arithmetic. H-B has 13 operations and runs 3.3 $\times$ slower than Sequential. Counting the ATen kernels those operations execute gives an equally good fit ($R^2 = 0.99$), so the effect is not an artifact of how we count. Adding the number of `nn.Module` calls as a second predictor explains almost all remaining variance ($R^2 = 0.998$; roughly 1.8 μ s per operation and 1.0 μ s per module call), including the Sequential/Modular gap. With six architectures and three fitted coefficients, we treat this decomposition as descriptive rather than predictive.

Compilation. `torch.compile` fuses each model’s operations into one compiled graph (Table I, column comp.). This narrows the batch-1 spread from 3.8 $\times$ to 1.6 $\times$ (20.9–33.2 μ s) and weakens the link to operator count ($R^2 = 0.65$), but it adds a fixed per-call cost of about 10 μ s. The three-operation models therefore become slower (Sequential 11.7 to 21.3 μ s), while the operator-heavy hybrids become faster (H-A 31.7 to 22.5 μ s). H-B remains the slowest model (1.6 $\times$ Sequential). For sub-kilobyte models at batch 1, compilation trades per-operator cost for per-call cost rather than giving a free speedup.

Batch size changes the ranking. Figure 5a shows two regimes. Up to a batch of about 64, latency is nearly flat and ordered by operator count. Past roughly 500 samples, per-

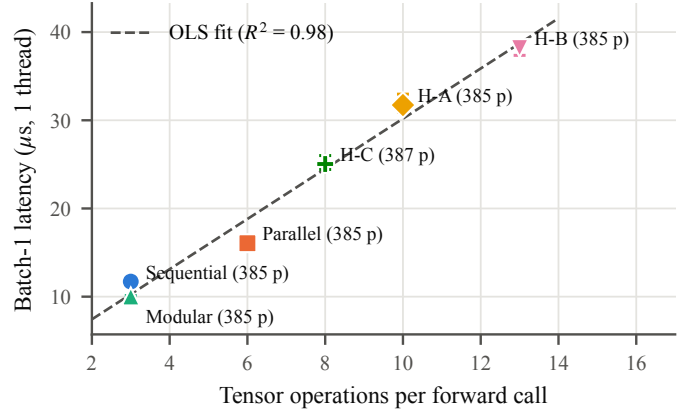


Fig. 4. Batch-1 inference latency vs. tensor operations per forward call. Parameter counts are in parentheses; error bars are 95% intervals across ten shuffled repeats. Latency is almost perfectly explained by the operator count ($R^2 = 0.98$) and is unrelated to the actual parameter count.

element work takes over and the ordering reshuffles. At batch 4096, H-C becomes the *fastest* model (156 μ s vs. 196 μ s for Sequential) despite having the most MACs. This is consistent with memory traffic mattering more than arithmetic: H-C’s widest activation is 20 values per sample, while Sequential’s is 128 and Parallel’s concatenation writes 128 more. The two models that build intermediate concatenated tensors are the slowest at batch 4096: H-B (322 μ s) and Parallel (314 μ s). The latency ranking of these architectures therefore depends on the batch size at which it is measured.

Threads. With four intra-op threads (Fig. 5b), batch-1 latency is unchanged. At larger batches the effect is inconsistent. Sequential improves at batch 4096 (196 to 146 μ s),

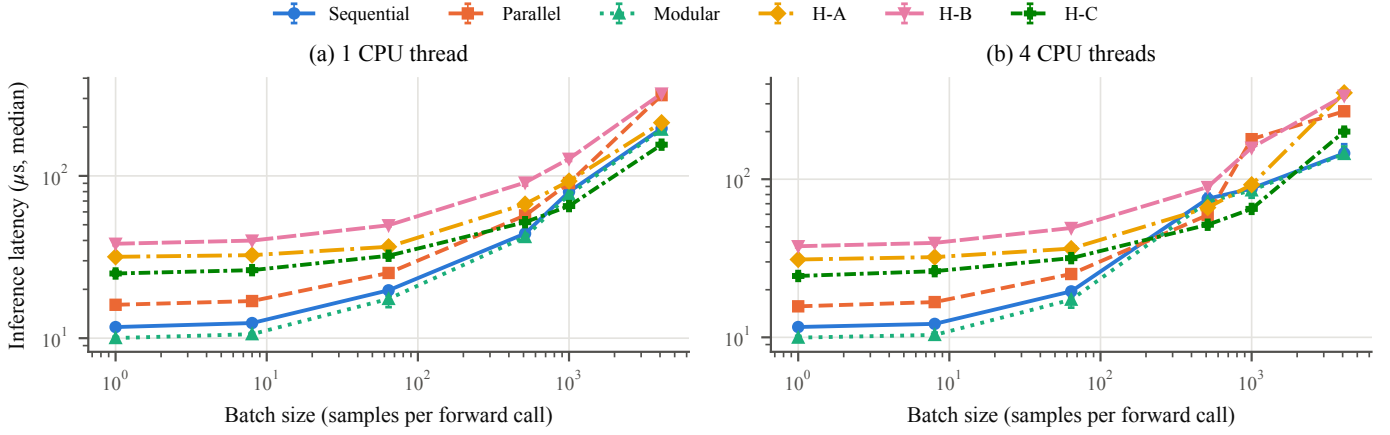


Fig. 5. Inference latency vs. batch size (median over 10 shuffled repeats of 1,000 calls; bars show the 10th–90th percentile of individual calls). (a) With one thread, the curves are flat up to batch ≈ 64 , where overhead dominates, and the ranking follows operator count. By batch 4096 it inverts toward the narrow-activation H-C. (b) Four threads rarely help models this small and sometimes hurt (Parallel at batch 1,000).

but Parallel at batch 1,000 slows from 90 to 179 μs and H-A at batch 4096 slows to 352 μs . For sub-kilobyte models, thread synchronization overhead can exceed the parallel work.

VI. DISCUSSION

A. Which Architecture Should a Practitioner Pick?

The results support a simple decision rule for small models on latency-sensitive paths.

- 1) *If the target has no known structure, keep it shallow and minimize operators.* On the linear and curved tasks every topology reached the noise floor, so the cheapest one, a single wide hidden layer with three operations, becomes the best choice. It trains fastest per step and has the lowest batch-1 latency.
- 2) *If the target is compositional and its stages are known, a staged modular hybrid is worth its overhead.* H-B cut chained-task excess error by $1.5\times$ at $1.81\times$ the training cost per step and $3.3\times$ the batch-1 latency, a modest accuracy gain (11%) at a real cost. For offline or batched inference, that trade can be worthwhile; for per-sample real-time loops it usually is not.
- 3) *Measure latency at the deployment batch size.* The ranking at batch 1 (operator-bound) and at batch 4096 (memory-bound) differ. A benchmark at one batch size can recommend the incorrect model for another.

B. Target Scaling Can Masquerade as Architecture

An earlier version of this study trained on unscaled targets, whose variance reaches 229 on the curved task. There, the single-paradigm models converged far more slowly than the hybrids (69 vs. 8 epochs on the curved task), and H-B appeared 43% better than Sequential on the chained task. Standardizing the targets removed nearly all of the convergence gap and shrank the chained-task advantage to 11%. Much of the apparent hybrid benefit was therefore an optimization effect: the additive, multi-path hybrids coped better with badly scaled

targets. This is a second example, after the pilot, of a protocol choice producing an apparent architecture effect.

C. Lessons for Small-Scale Benchmarking

The pilot study shows how easily a benchmark can produce a confident, wrong conclusion. A few practices catch such errors and are cheap enough for any student or practitioner to adopt. First, include a *null control*: a pair that should show no difference, such as our identical Sequential and Modular networks. Ours served several roles: it exposed the pilot’s framework confound, flagged CPU contention during training, and revealed module-call overhead that operation counts alone miss. Second, hold the framework and training loop fixed so that timing measures the model and not the software stack [11]. Third, check accuracy against trivial baselines: a model whose loss equals the target variance has learned nothing [25]. Fourth, standardize targets before comparing training dynamics, so that optimization difficulty is not mistaken for an architecture effect.

D. Limitations and Threats to Validity

Scale. The models are tiny (≈ 385 parameters) and the tasks are one-dimensional. This is deliberate, as it isolates the topology from overall capacity and makes the five-seed run cheap. It also means the absolute latencies are dominated by per-operator overhead in a way that will shrink, though not vanish, for larger models, where compute and memory traffic take over [10]. The ranking of architectures at batch 4096, where arithmetic begins to matter, is a better guide to larger-scale behavior than batch 1.

Hardware and runtime. All timings come from one Apple M5 CPU. We measured eager PyTorch and `torch.compile`; other runtimes (ONNX Runtime, TensorRT) and other hardware (GPUs, microcontrollers, neural accelerators) have different overhead profiles. We therefore present latency *trends* across architectures, not absolute numbers to be quoted for other platforms.

Task design. The chained task was written by the same author who designed H-B, so H-B’s structural match to it is an intentional best case for modular inductive bias, not an unbiased test. We do not claim that staged modularity helps on arbitrary compositional problems, only that it *can* when the decomposition is known.

Statistics. Five seeds detect large effects but not small ones. We report Welch’s *t*-tests without multiple-comparison correction, so isolated marginal results should be read as suggestive. Hyperparameters (learning rate, epochs, batch size) were fixed for all models rather than tuned per architecture, which may disadvantage architectures that prefer other settings.

VII. CONCLUSION

We benchmarked six neural architectures that share a 385-parameter budget and differ only in how those parameters are arranged. Three findings stand out. First, configuration changes final accuracy only when the target has structure for it to match. A staged modular hybrid reduced error on a compositional task by 11%, a gap that held after 300 epochs, while every other structure tied on simple tasks. Second, small-model latency is decided by operator count at small batch sizes and by activation width at large ones, and parameter count predicts neither; `torch.compile` narrows the operator-count penalty but adds a fixed per-call cost. Third, protocol choices can masquerade as architecture effects: a cross-framework confound in our pilot and unscaled targets in an earlier run both produced large apparent advantages that disappeared under a controlled protocol.

Future work will extend the study to other runtimes (ONNX Runtime) and hardware (GPUs, neural accelerators, micro-controllers), to larger parameter budgets where arithmetic dominates overhead, and to real-world tasks where the stage decomposition must be learned rather than designed.

ACKNOWLEDGMENT

The pilot study used Google Colab. AI tools assisted with experiment code, figures, and drafting; the author verified all content.

REFERENCES

- [1] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [2] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, “Aggregated residual transformations for deep neural networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5987–5995.
- [3] R. Caruana, “Multitask learning,” *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997.
- [4] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017, arXiv:1701.06538.
- [5] R. Eldan and O. Shamir, “The power of depth for feedforward neural networks,” in *Proceedings of the 29th Conference on Learning Theory (COLT)*, ser. Proceedings of Machine Learning Research, vol. 49, 2016, pp. 907–940, arXiv:1512.03965.
- [6] Z. Lu, H. Pu, F. Wang, Z. Hu, and L. Wang, “The expressive power of neural networks: A view from the width,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, arXiv:1709.02540.
- [7] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, “ShuffleNet V2: Practical guidelines for efficient CNN architecture design,” in *Computer Vision – ECCV 2018*, ser. Lecture Notes in Computer Science. Springer, 2018, pp. 122–138.
- [8] M. Dehghani, A. Arnab, L. Beyer, A. Vaswani, and Y. Tay, “The efficiency misnomer,” in *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022, arXiv:2110.12894.
- [9] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Thakker, A. Torrini, P. Warden, J. Cordaro, G. Di Guglielmo, J. Duarte, S. Gibellini, V. Parekh, H. Tran, N. Tran, N. Wenxu, and X. Xuesong, “MLPerf tiny benchmark,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021, arXiv:2106.07597.
- [10] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [11] S. Shi, Q. Wang, P. Xu, and X. Chu, “Benchmarking state-of-the-art deep learning software tools,” in *Proceedings of the 7th International Conference on Cloud Computing and Big Data (CCBD)*, 2016, pp. 99–104.
- [12] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Sepah, E. Raff, K. Madan, V. Voleti, S. E. Kahou, V. Michalski, D. Serdyuk, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent, “Accounting for variance in machine learning benchmarks,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 747–769.
- [13] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.
- [14] S. Zagoruyko and N. Komodakis, “Wide residual networks,” in *Proceedings of the British Machine Vision Conference (BMVC)*, 2016, pp. 87.1–87.12.
- [15] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, arXiv:1612.01474.
- [16] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [17] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein, “Neural module networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 39–48.
- [18] D. Hupkes, V. Dankers, M. Mul, and E. Bruni, “Compositionality decomposed: How do neural networks generalise?” *Journal of Artificial Intelligence Research*, vol. 67, pp. 757–795, 2020.
- [19] J. Pfeiffer, S. Ruder, I. Vulić, and E. M. Ponti, “Modular deep learning,” *Transactions on Machine Learning Research*, 2023, arXiv:2302.11529.
- [20] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [21] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2261–2269.
- [22] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, “MnasNet: Platform-aware neural architecture search for mobile,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2815–2823.
- [23] V. J. Reddi et al., “MLPerf inference benchmark,” in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.
- [24] C. Coleman, D. Kang, D. Narayanan, L. Nardi, T. Zhao, J. Zhang, P. Bailis, K. Olukotun, C. Ré, and M. Zaharia, “Analysis of DAWN-Bench, a time-to-accuracy machine learning performance benchmark,” *ACM SIGOPS Operating Systems Review*, vol. 53, no. 1, pp. 14–25, 2019.
- [25] Z. C. Lipton and J. Steinhardt, “Troubling trends in machine learning scholarship,” *Queue*, vol. 17, no. 1, pp. 45–77, 2019.
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, arXiv:1912.01703.

- [27] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015, arXiv:1412.6980.